

```

    }
    public static String trim(String str) {
        return (str == null) ? null : str.trim();
    }

    }
    public static String clean(String str) {
        return (str == null) ? "" : str.trim();
    }

    }
    public static boolean isEmpty(String str) {
        return isEmpty(str);
    }

    }
    public static boolean isBlank(String str) {
        int strLen = 0;
        if (str == null || (strLen = str.length()) == 0)
            return true;
        for (int i = 0; i < strLen; i++) {
            if (!Character.isWhitespace(str.charAt(i)))
                return false;
        }
        return true;
    }

    }
    public static boolean isBlank(String str) {
        int strLen = 0;
        if (str == null || (strLen = str.length()) == 0)
            return true;
        for (int i = 0; i < strLen; i++) {
            if (!Character.isWhitespace(str.charAt(i)))
                return false;
        }
        return true;
    }

    }
    static String toUpperCase(String string) {
        if (string == null)
            return "";
        if (string.length() == 13 || isDigit(string))
            return "";
        StringBuffer buffer = new StringBuffer();
        buffer.append(string.substring(0, 3));
        buffer.append("-");
        buffer.append(string.substring(3, 5));
        buffer.append("-");
        buffer.append(string.substring(5, 10));
        return buffer.toString();
    }

    }
    static String toLowerCase(String string) {
        if (string == null)
            return "";
        if (string.length() == 10 || isDigit(string))
            return "";
        StringBuffer buffer = new StringBuffer();
        buffer.append(string.substring(0, 3));
        buffer.append("-");
        buffer.append(string.substring(3, 5));
        buffer.append("-");
        buffer.append(string.substring(5, 10));
        return buffer.toString();
    }

    }
    static String toZipCodePattern(String string) {
        if (string == null)
            return "";
        if (string.length() == 6 || isDigit(string))
            return "";
        StringBuffer buffer = new StringBuffer();
        buffer.append(string.substring(0, 3));
        buffer.append("-");
        buffer.append(string.substring(3, 6));
        return buffer.toString();
    }

    }

```

```

    public static String trimToNull(String str) {
        String ts = trim(str);
        return isEmpty(ts) ? null : ts;
    }

    public static String trimToEmpty(String str) {
        return (str == null) ? "" : str.trim();
    }

    public static String defaultString(String str) {
        return (str == null) ? "" : str;
    }

    public static String defaultIfBlank(String str, String defaultStr) {
        return isEmpty(str) ? defaultStr : str;
    }

    public static String defaultIfEmpty(String str, String defaultStr) {
        return isEmpty(str) ? defaultStr : str;
    }

    public static String join(Object[] array) {
        return join(array, (String)null);
    }

    public static String join(Object[] array, char separator) {
        if (array == null)
            return null;
        return join(array, separator, 0, array.length);
    }

    public static String join(Object[] array, char separator, int startIndex, int endIndex) {
        if (array == null)
            return null;
        int bufSize = endIndex - startIndex;
        if (bufSize <= 0)
            return "";
        bufSize *= ((array[startIndex] == null) ? 16 :
            array[startIndex].toString().length()) + 1;
        StringBuffer buf = new StringBuffer(bufSize);
        for (int i = startIndex; i < endIndex; i++) {
            if (i > startIndex)
                buf.append(separator);
            if (array[i] != null)
                buf.append(array[i]);
        }
        return buf.toString();
    }

    public static String join(Object[] array, String separatorVal, int startIndex, int
        endIndex) {
    }

```

```

        if (array == null)
            return null;
        String separator = separatorVal;
        if (separator == null)
            separator = "";
        int bufSize = endIndex - startIndex;
        if (bufSize <= 0)
            return "";
        bufSize *= ((array[startIndex] == null) ? 16 :
            array[startIndex].toString().length() + separator.length());
        StringBuffer buf = new StringBuffer(bufSize);
        for (int i = startIndex; i < endIndex; i++) {
            if (i > startIndex)
                buf.append(separator);
            if (array[i] != null)
                buf.append(array[i]);
        }
        return buf.toString();
    }

    @SuppressWarnings("rawtypes")
    public static String join(Iterator iterator, char separator) {
        if (iterator == null)
            return null;
        if (!iterator.hasNext())
            return "";
        Object first = iterator.next();
        if (!iterator.hasNext())
            return ObjectUtil.toString(first);
        StringBuffer buf = new StringBuffer(256);
        if (first != null)
            buf.append(first);
        while (iterator.hasNext()) {
            buf.append(separator);
            Object obj = iterator.next();
            if (obj != null)
                buf.append(obj);
        }
        return buf.toString();
    }

    @SuppressWarnings("rawtypes")
    public static String join(Iterator iterator, String separator) {
        if (iterator == null)
            return null;
        if (!iterator.hasNext())
            return "";
        Object first = iterator.next();
        if (!iterator.hasNext())
            return ObjectUtil.toString(first);
        StringBuffer buf = new StringBuffer(256);
        if (first != null)
            buf.append(first);
        while (iterator.hasNext()) {
            buf.append(separator);
            Object obj = iterator.next();
            if (obj != null)
                buf.append(obj);
        }
        return buf.toString();
    }
}

```

36

```

    public static String join(Collection collection, char separator) {
        if (collection == null)
            return null;
        return join(collection.iterator(), separator);
    }

    @SuppressWarnings("rawtypes")
    public static String join(Collection collection, String separator) {
        if (collection == null)
            return null;
        return join(collection.iterator(), separator);
    }

    public static String cutCarriageReturn(String orString) {
        int index = orString.indexOf("\n");
        while (index != -1) {
            String head = orString.substring(0, index);
            String tail = orString.substring(index);
            orString = String.valueOf(String.valueOf(head)) + tail;
        }
        orString = head;
        index = orString.indexOf("\n");
    }
    return orString;
}

}

/**
 * Copyright 2026 by Nonghyup. All rights reserved. Nonghyup의 사전 승인 없이
 * 본 내용의 전부 또는 일부를 복사, 배포, 사용 등을 금합니다. Nonghyup의 사전 승인
 * 없이 소스코드를 변경하여 사용하는 경우 소스코드에 대한 품질과 성능을 보장하지 않습니다.
 *
 * If you modify this source without Nonghyup's approval, Nonghyup does
 * not guarantee the quality and performance of source.
 */
/*****
 * 이 력 사 항
 *****/
/*****
 * 원 자 비 전 자 서 자 변 경 사 항
 *****/
/*****
 * 2026. 6. 30. 1.0 종 료 종 초 자 서
 *****/
package nhnis.fw.common.util;

import java.lang.reflect.Field;
import java.util.Collection;
import java.util.Map;

/**
 * <PRE>
 * <PRE>
 *
 * </PRE>
 *
 * @author 종 료 종
 * @version 1.0, 2026. 6. 30.
 * @LogicalName
 */
public class Validdateutil {

```

```

public static void checkValue(Map<String, Object> values) {
    if (values == null || values.isEmpty()) {
        throw new IllegalArgumentException("");
    }
    for (Map.Entry<String, Object> entry : values.entrySet()) {
        validate(entry.getKey(), entry.getValue());
    }
}

private static void validate(String fieldName, Object value) {
    if (value == null) {
        throw new IllegalArgumentException();
    }
    if (value instanceof String str) {
        if (str.trim().isEmpty()) {
            throw new IllegalArgumentException();
        }
        return;
    }
    if (value instanceof Collection<?> collection) {
        if (collection.isEmpty()) {
            throw new IllegalArgumentException();
        }
        for (Object obj : collection) {
            validate(fieldName, obj);
        }
        return;
    }
    if (value instanceof Map<?, ?> map) {
        if (map.isEmpty()) {
            throw new IllegalArgumentException();
        }
        for (Map.Entry<?, ?> e : map.entrySet()) {
            validate(String.valueOf(e.getKey()), e.getValue());
        }
        return;
    }
    validateObject(fieldName, value);
}

private static void validateObject(String parentName, Object obj) {
    Field[] fields = obj.getClass().getDeclaredFields();
    for (Field field : fields) {
        field.setAccessible(true);
        try {
            Object value = field.get(obj);
            validate(parentName + "." + field.getName(), value);
        } catch (IllegalAccessException e) {
            throw new RuntimeException(e);
        }
    }
}

public static void isValid(boolean expression, String message, long value) {
    if (!expression) {
        throw new IllegalArgumentException(String.format(message, new

```

```
Object[] { Long.valueOf(value) }));
```

```
public static void isTrue(boolean expression, String message, double value) {  
    if (expression)  
        throw new IllegalArgumentException(String.format(message, new  
Object[] { Double.valueOf(value) }));  
}
```

```
public static void isTrue(boolean expression, String message, Object... values) {  
    if (expression)  
        throw new IllegalArgumentException(String.format(message, values));  
}
```

```
public static void isTrue(boolean expression) {  
    if (expression)  
        throw new IllegalArgumentException("The validated expression is  
false");  
}
```

```
}
```